POLITECHNIKA ŚWIĘTOKRZYSKA WYDZIAŁ ELEKTRONIKI, AUTOMATYKI I INFORMATYKI	
Programowanie obiektowe 2 - Projekt	
Temat: Gra Space Invaders	
Autorzy: Kajetan Gołucki, Marcin Hajduk	
Rok studiów: 2024/2025	Grupa dziekańska: 2ID12A

1.Ogólny opis projektu

Projekt „Gra Space Invaders” to klasyczna gra zręcznościowa napisana w języku Java bazująca na orginalnej grze Space Invaders. Celem gry jest zestrzelenie fal przeciwników, unikanie ich pocisków oraz ochrona własnego statku za pomocą bunkrów. Gra zawiera podstawowe mechanizmy, takie jak: Poruszanie się gracza, Strzelanie pociskami, Fale przeciwników, System punktów i żyć.

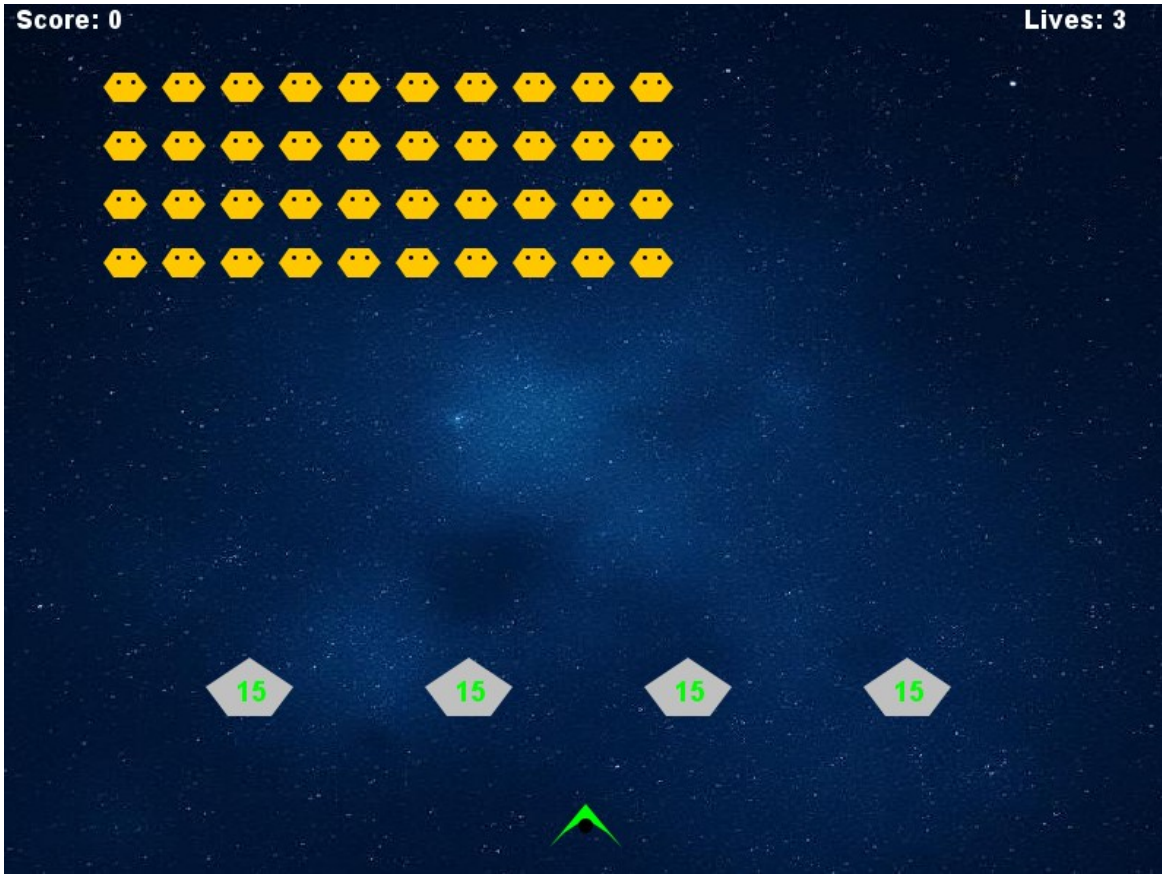
Użyte technologie, frameworki i biblioteki: Cały projekt został stworzony jako projekt Maven, napisany w języku Java, wykorzystując jego obiektowość i biblioteki standardowe: Swing – biblioteka do tworzenia interfejsu graficznego, javax.imageio.ImageIO – do ładowania obrazów, javax.sound.sampled – do odtwarzanie dźwięków, JUnit – framework wykorzystywany do testów jednostkowych.

2.Informacje na temat funkcjonalności projektu

Cała gra bazuje na orginalnej grze space invaders, więc ogólna funkcjonalność jest do niej bardzo zbliżona.

Funkcjonalność:

- **Sterowanie graczem:** Gracz porusza statkiem kosmicznym przy użyciu klawiszy strzałek (lewo/prawo) oraz strzela, wciskając klawisz spacji.
- **Ruch przeciwników:** Wrogowie poruszają się poziomo, zmieniają kierunek po osiągnięciu krawędzi ekranu i przesuwają się w dół, aż do osiągnięcia poziomu bunkrów.
- **System pocisków:** Gra obsługuje pociski zarówno gracza, jak i przeciwników. Pociski są usuwane, gdy opuszczą ekran lub po kolizji.
- **Kolizje:** Realizowana jest detekcja kolizji między pociskami a przeciwnikami i bunkrami, oraz między pociskami przeciwników a bunkrami i statkiem gracza.
- **Bunkry:** Utrudniają graczowi zdobywanie punktów, i chronią go przed atakami przeciwników.
- **Mechanika gry:** Gra monitoruje punkty, liczbę żyć gracza oraz najwyższy wynik, zapisując go do pliku.
- **Dźwięk:** W tle odtwarzana jest muzyka, którą można włączać i wyłączać.
- **Tryby gry:** Obsługa pauzy (klawisz 'P'), restartu gry po przegranej (klawisz 'R') oraz przejścia do kolejnych fal przeciwników.



3.Uruchamianie oraz obsługa projektu

Kompilacja i uruchomienie: Projekt można uruchomić w środowisku IDE, takim jak IntelliJ IDEA.

Obsługa gry:

- **Ruch statkiem**: Użyj klawiszy strzałka w lewo i strzałka w prawo.
- **Strzelanie:** Naciśnij spację.
- **Pauza:** Naciśnij P, aby zatrzymać lub wznowić grę.
- **Restart:** Po zakończeniu gry (GAME OVER) naciśnij R, aby rozpocząć nową rundę.
- **Muzyka:** Naciśnij M, aby włączyć lub wyłączyć muzykę w tle.

Testowanie: Testy jednostkowe napisane przy użyciu **JUnit** pozwalają zweryfikować działanie kluczowych komponentów gry, takich jak ruch przeciwników, kolizje, strzelanie oraz obsługa zdarzeń klawiatury.

4.Struktura projektu

Projekt jest podzielony na dwa główne pakiety:

spaceinvaders - Zawiera klasy związane z logiką gry:

- Player – sterowanie statkiem gracza.
- Enemy – przeciwnicy i ich zachowanie.
- Bullet, EnemyBullet – pociski gracza i przeciwników.
- Bunker – bunkry chroniące gracza.
- Game – główna logika gry.

Tests - Zawiera testy jednostkowe JUnit 5 dla kluczowych klas:

- GameTest – testy logiki gry (ruch przeciwników, kolizje, reset gry).
- PlayerTest – testy ruchu i inicjalizacji gracza.

Klasa Player: Reprezentuje gracza, jest odpowiedzialna za przechowywanie informacji o pozycji, rozmiarach oraz rysowaniu statku gracza. Implementuje również logikę poruszania się gracza po ekranie.

Pola klasy:

- **x, y:** Przechowują współrzędne statku gracza na ekranie.
- **width, height:** Określają szerokość i wysokość statku gracza.
- **SPEED:** Stała określająca prędkość poruszania się statku gracza

Metody klasy:

Konstruktor Player(int x, int y): Inicjalizuje obiekt gracza, ustawiając jego pozycję na podstawie przekazanych współrzędnych x i y. Szerokość i wysokość statku są domyślnie ustawione na 50 i 30 pikseli.

void draw(Graphics g): Rysuje statek gracza na ekranie przy użyciu obiektu Graphics.

void moveLeft(): Przesuwa statek gracza w lewo o wartość SPEED.

void moveRight(): Przesuwa statek gracza w prawo o wartość SPEED.

Rectangle getBounds(): Zwraca prostokąt reprezentujący granice statku gracza, metoda używana do wykrywania kolizji.

int getX(), int getY(), int getWidth(): Metody zwracające odpowiednio współrzędną x, współrzędną y oraz szerokość statku gracza. Są one używane do zarządzania pozycją i rozmiarami statku w innych częściach programu.

Klasa Bullet: Reprezentuje pocisk wystrzeliwany przez gracza, jest odpowiedzialna za przechowywanie informacji o pozycji, rozmiarach, stanie aktywności oraz rysowaniu pocisku na ekranie. Implementuje również logikę poruszania się pocisku.

Pola klasy:

- **x, y:** Przechowują współrzędne pocisku na ekranie.
- **WIDTH, HEIGHT:** Stałe określające szerokość i wysokość pocisku
- **SPEED:** Stała określająca prędkość poruszania się pocisku
- **active:** Flaga określająca, czy pocisk jest aktywny (czy znajduje się na ekranie).

Metody klasy:

Konstruktor Bullet(int startX, int startY): inicjalizuje obiekt pocisku, ustawiając jego pozycję na podstawie przekazanych współrzędnych startX i startY.

void move(): przesuwa pocisk do góry o wartość SPEED. Jeśli pocisk opuści ekran (y < 0), flaga active jest ustawiana na false, co oznacza, że pocisk przestaje być aktywny.

void draw(Graphics g): rysuje pocisk na ekranie przy użyciu obiektu Graphics. Pocisk jest reprezentowany jako prostokąt wypełniony kolorem żółtym.

boolean isActive(): zwraca wartość flagi active, która określa, czy pocisk jest aktywny.

void setActive(boolean active): ustawia wartość flagi active. Metoda ta może być używana do aktywowania lub deaktywowania pocisku.

Rectangle getBounds(): zwraca prostokąt (Rectangle) reprezentujący granice pocisku, metoda używana do wykrywania kolizji z innymi obiektami w grze.

Klasa Bunker: reprezentuje bunkier (osłonę) w grze Space Invaders. Bunkier służy jako ochrona dla gracza przed pociskami wrogów. Klasa jest odpowiedzialna za przechowywanie informacji o pozycji, rozmiarach, wytrzymałości oraz rysowaniu bunkra na ekranie. Implementuje również logikę zmniejszania wytrzymałości bunkra po trafieniu.

Pola klasy:

- **x, y:** Przechowują współrzędne bunkra na ekranie.
- **width, height:** Określają szerokość i wysokość
- **health:** Przechowuje wytrzymałość bunkra

Metody klasy:

Konstruktor Bunker(int x, int y): Inicjalizuje obiekt bunkra, ustawiając jego pozycję, szerokość, wysokość i wytrzymałość są ustawiane na domyślne wartości.

void draw(Graphics g): Rysuje bunkier na ekranie przy użyciu obiektu Graphics. Bunkier jest reprezentowany jako wielokąt o kształcie przypominającym tarczę. Wypełniony jest kolorem szarym, na środku bunkra wyświetlana jest aktualna wartość wytrzymałości w kolorze zielonym.

Rectangle getBounds(): Zwraca prostokąt reprezentujący granice bunkra. Metoda ta jest używana do wykrywania kolizji z pociskami gracza lub wrogów.

boolean takeDamage(): Zmniejsza wytrzymałość bunkra o 1 przy każdym wywołaniu. Jeśli wytrzymałość spadnie do 0 lub poniżej, metoda zwraca true, co oznacza, że bunkier został zniszczony. W przeciwnym razie zwraca false.

Klasa Enemy: Reprezentuje przeciwnika, jest odpowiedzialna za przechowywanie informacji o pozycji, rozmiarach oraz rysowaniu przeciwnika na ekranie. Przeciwnik jest reprezentowany jako kosmita.

Pola klasy:

- **x, y:** Przechowują współrzędne przeciwnika na ekranie.
- **width, height:** Określają szerokość i wysokość przeciwnika (domyślnie 30 i 20 pikseli).

Metody klasy:

Konstruktor Enemy(int x, int y): Inicjalizuje obiekt przeciwnika, ustawiając jego pozycję na podstawie przekazanych współrzędnych x i y.

void draw(Graphics g): Rysuje przeciwnika na ekranie przy użyciu obiektu Graphics. Przeciwnik jest reprezentowany jako wielokąt o kształcie przypominającym kosmitę. Wypełniony jest kolorem pomarańczowym. Dodatkowo na głowie przeciwnika rysowane są dwa czarne kółka symulujące oczy.

int getX(), int getY(): Zwracają współrzędne x i y przeciwnika.

int getWidth(), int getHeight(): Zwracają szerokość i wysokość przeciwnika.

void setX(int x), void setY(int y): Ustawiają nową wartość współrzędnej x lub y przeciwnika, są używane do aktualizacji pozycji przeciwnika podczas poruszania się po ekranie.

Klasa EnemyBullet: reprezentuje pocisk wystrzeliwany przez przeciwnika, jest odpowiedzialna za przechowywanie informacji o pozycji, rozmiarach, stanie aktywności oraz rysowaniu pocisku na ekranie. Implementuje również logikę poruszania się pocisku w dół ekranu.

Pola klasy:

- **x, y:** Przechowują współrzędne pocisku na ekranie.
- **WIDTH, HEIGHT:** Stałe określające szerokość i wysokość pocisku.
- **SPEED:** Stała określająca prędkość poruszania się pocisku.
- **active:** Flaga określająca, czy pocisk jest aktywny.

Metody klasy:

Konstruktor EnemyBullet(int startX, int startY): Inicjalizuje obiekt pocisku, ustawiając jego pozycję na podstawie przekazanych współrzędnych startX i startY.

void move(): Przesuwa pocisk w dół o wartość SPEED. Jeśli pocisk opuści ekran, flaga active jest ustawiana na false, co oznacza, że pocisk przestaje być aktywny.

void draw(Graphics g): Rysuje pocisk na ekranie przy użyciu obiektu Graphics. Pocisk jest reprezentowany jako prostokąt wypełniony kolorem czerwonym.

boolean isActive(): Zwraca wartość flagi active

void setActive(boolean active) : Ustawia wartość flagi active. Metoda ta może być używana do aktywowania/deaktywowania pocisku.

Rectangle getBounds(): Zwraca prostokąt reprezentujący granice pocisku, jest używana do wykrywania kolizji z innymi obiektami w grze, takimi jak gracz lub bunkier.

Klasa Game: Jest główną klasą gry Space Invaders, odpowiedzialną za zarządzanie logiką gry, obsługę zdarzeń, rysowanie komponentów oraz kontrolowanie stanu gry. Dziedziczy po JPanel i implementuje interfejs KeyListener, co umożliwia obsługę klawiatury.

Pola klasy:

- **WIDTH, HEIGHT:** Stałe określające szerokość i wysokość okna gry (domyślnie 800x600 pikseli).
- **SHOOT_COOLDOWN:** Stała określająca czas odczekiwania między strzałami gracza (domyślnie 500 ms).
- **enemies:** Lista przechowująca obiekty przeciwników (Enemy).
- **bullets:** Lista przechowująca pociski gracza (Bullet).
- **enemyBullets:** Lista przechowująca pociski przeciwników (EnemyBullet).
- **bunkers:** Lista przechowująca bunkry (Bunker).
- **player:** Obiekt reprezentujący gracza (Player).
- **score:** Aktualny wynik gracza.
- **highScore:** Najwyższy wynik osiągnięty w grze.
- **isGameOver:** Flaga określająca, czy gra została zakończona.
- **isPaused:** Flaga określająca, czy gra jest zapauzowana.
- **isMusicOn:** Flaga określająca, czy muzyka w tle jest włączona.
- **enemyDirection:** Kierunek poruszania się przeciwników (1 - w prawo, -1 - w lewo).
- **lives:** Liczba żyć gracza (domyślnie 3).
- **lastShotTime:** Czas ostatniego strzału gracza.
- **backgroundMusic:** Obiekt przechowujący muzykę w tle.
- **enemyTimer, bulletTimer, repaintTimer, enemyShootTimer, enemyBulletMoveTimer:** Timery kontrolujące różne aspekty gry.
- **backgroundImage:** Obraz tła gry.

Metody klasy:

Konstruktor Game(): ustawia wszystkie podstawowe elementy gry

Wczytuje obraz tła, inicjalizuje gracza, przeciwników, pociski i bunkry. Uruchamia główną pętlę gry oraz odtwarza muzykę w tle.

void loadHighScore(): Wczytuje najwyższy wynik z pliku highscore.txt.

void saveHighScore(): Zapisuje najwyższy wynik do pliku highscore.txt.

void initializeEnemies(): Tworzy i ustawia przeciwników w rzędach i kolumnach na początku gry.

void initializeBullets(): Inicjalizuje listę pocisków gracza.

void initializeBunkers(): Tworzy i ustawia bunkry na planszy.

void togglePause(): Włącza lub wyłącza pauzę w grze, zatrzymując lub wznawiając timery.

void toggleMusic(): Włącza lub wyłącza muzykę w tle.

void runGameLoop(): Uruchamia główną pętlę gry, inicjalizując timery odpowiedzialne za ruch przeciwników, pocisków, odświeżanie ekranu oraz strzelanie przeciwników.

void moveEnemies(): Przesuwa przeciwników w odpowiednim kierunku, zmienia kierunek i przesuwa ich w dół gdy dotrą do krawędzi ekranu.

void shoot(): Tworzy nowy pocisk gracza, jeśli minął odpowiedni czas od ostatniego strzału.

void moveBullets(): Przesuwa pociski gracza i usuwa te które opuściły ekran.

void enemyShoot(): Losowo wybiera przeciwników, którzy strzelają w kierunku gracza.

void moveEnemyBullets(): Przesuwa pociski przeciwników i usuwa te, które opuściły ekran.

void checkCollisions(): Sprawdza kolizje pocisków gracza z przeciwnikami i aktualizuje wynik.

void checkBulletBunkerCollisions(): Sprawdza kolizje pocisków z bunkrami i usuwa zniszczone bunkry.

void checkPlayerHit(): Sprawdza, czy gracz został trafiony przez pocisk przeciwnika, i odejmuje życie.

void nextWave(): Przechodzi do następnej fali przeciwników, resetując ich pozycje i odradzając bunkry.

void resetGame(): Resetuje stan gry i rozpoczyna nową rundę.

void playBackgroundMusic(): Odtwarza muzykę w tle w pętli.

void paintComponent(Graphics g): Rysuje wszystkie komponenty gry na ekranie, w tym gracza, przeciwników, pociski, bunkry oraz informacje o wyniku i życiach.

void keyPressed(KeyEvent e): Obsługuje zdarzenia naciśnięcia klawiszy, takie jak poruszanie się gracza, strzelanie, restartowanie gry, pauzowanie i włączanie/wyłączanie muzyki.

void keyReleased(KeyEvent e), void keyTyped(KeyEvent e): Metody wymagane przez interfejs KeyListener, ale nieużywane w tej implementacji.

public static void main(String[ ] args): Metoda główna uruchamiająca grę, tworząca okno i inicjalizująca instancję gry.

Metody pomocnicze:
getEnemies(), getBullets(), getLives(), getScore(), getPlayer(), isGameOver(), getEnemyBullets(), isPaused(): Zwracają różne informacje o stanie gry.

5.Podział pracy w zespole

	Kajetan Gołucki	Marcinn Hajduk
KOD PROGRAMU	60%	40%
TESTY	50%	50%
DESIGN/MUZYKA	30%	70%
DOKUMENTACJA	80%	20%
SPRAWOZDANIE	60%	40%